

Práctica: Ingeniería Inversa con EF Core (Database-First)

Paso a paso — Generar modelos de dominio y DbContext desde SQL Server en un entorno Windows

Curso	Técnico de Desarrollo Full Stack con C# y Angular
Sesión	Sesión 4 — Persistencia con Entity Framework Core
Docente	Edwin Barrientos
Duración estimada	60–90 minutos
Nivel	Básico – Intermedio (Database-First en Windows)
Entorno	Windows 10/11 · SQL Server 2019/2022 o Express · SSMS · Visual Studio 2022 · .NET 8 SDK

1. Objetivos

- Ejecutar los scripts DDL y DML para aprovisionar la base de datos **TienditaDB** (5 tablas relacionales) en Microsoft SQL Server.
- Crear un proyecto de aplicación en C# (.NET 8) dentro de Visual Studio 2022 en un entorno Windows nativo.
- Instalar los paquetes NuGet necesarios para el proveedor SQL Server y herramientas de scaffolding de EF Core.
- Ejecutar la ingeniería inversa (Reverse Engineering) con Scaffold-DbContext (PowerShell / PMC) o la CLI `dotnet ef`.
- Analizar e interpretar las entidades autogeneradas, las propiedades de navegación y el archivo DbContext.
- Desacoplar la cadena de conexión hacia `appsettings.json` y validar la integridad con consultas LINQ con `Include`.

2. Desarrollo paso a paso

Paso 1 — Ejecutar los scripts SQL (DDL y DML) en SQL Server (Windows)

Para iniciar la práctica, se requiere desplegar la base de datos de la **tiendita** en su instancia local de SQL Server. Los archivos SQL incluidos en la carpeta `TienditaBD/` contienen la estructura y datos de 5 tablas: `Categorias`, `Proveedores`, `Productos`, `Ventas` y `DetalleVentas`.

Opción A: Ejecución mediante SQL Server Management Studio (SSMS)

- Abra **SQL Server Management Studio (SSMS)** e inicie sesión en su instancia local (`.`, `localhost` o `.\SQLEXPRESS`) con Autenticación de Windows.
- Haga clic en **Archivo > Abrir > Archivo...** (o presione `Ctrl + O`) y seleccione `Tiendita_Completo.sql`.
- Haga clic en **Ejecutar** o presione la tecla **F5**. Verifique en la pestaña *Resultados* que las 5 tablas han sido creadas y pobladas.

Opción B: Ejecución mediante consola de Windows (PowerShell / CMD)

```
# Ejecutar en PowerShell apuntando a la instancia SQL Server local
sqlcmd -S ".\SQLEXPRESS" -E -i "TienditaBD\Tiendita_Completo.sql"
# Si utiliza la instancia por defecto Developer/Enterprise:
sqlcmd -S "localhost" -E -i "TienditaBD\Tiendita_Completo.sql"
```

Verificación de integridad: En el Explorador de Objetos de SSMS, expanda *Databases > TienditaDB > Tables*. Debe observar las tablas `dbo.Categorias`, `dbo.Proveedores`, `dbo.Productos`, `dbo.Ventas` y `dbo.DetalleVentas` con sus respectivas Primary Keys y Foreign Keys.

Paso 2 — Crear el proyecto en Visual Studio 2022 (Windows)

A continuación, crearemos el proyecto en C# que consumirá la base de datos existente mediante Entity Framework Core 8.

Creación desde la interfaz gráfica de Visual Studio 2022:

- Inicie **Visual Studio 2022** y seleccione **Crear un nuevo proyecto**.
- En el filtro de plantillas, elija lenguaje **C#**, plataforma **Windows** y tipo de proyecto **Consola**. Seleccione *Aplicación de consola* y presione *Siguiente*.
- **Nombre del proyecto:** TienditaApp. Ubique el proyecto en su directorio de trabajo habitual.
- **Marco de trabajo:** Seleccione **.NET 8.0 (Compatibilidad con soporte a largo plazo)**. Haga clic en *Crear*.

Alternativa rápida mediante Windows PowerShell / Terminal:

```
# Crear la aplicación de consola en .NET 8
dotnet new console -n TienditaApp -f net8.0
cd TienditaApp
```

Paso 3 — Instalar los paquetes NuGet requeridos de EF Core

Para realizar la ingeniería inversa hacia SQL Server, se necesitan 3 paquetes fundamentales del ecosistema EF Core:

Paquete NuGet	Propósito en el entorno Windows
Microsoft.EntityFrameworkCore.SqlServer	Proveedor oficial de EF Core para conectarse y traducir LINQ a T-SQL de SQL Server.
Microsoft.EntityFrameworkCore.Design	Lógicas de compilación y generación de código usadas por las herramientas de diseño.
Microsoft.EntityFrameworkCore.Tools	Habilita los cmdlets de PowerShell (como Scaffold-DbContext) en la PMC de Visual Studio.

Método 1: Consola del Administrador de Paquetes (PMC) en Visual Studio

Vaya al menú superior: **Herramientas > Administrador de paquetes NuGet > Consola del Administrador de paquetes** y ejecute:

```
Install-Package Microsoft.EntityFrameworkCore.SqlServer
Install-Package Microsoft.EntityFrameworkCore.Design
Install-Package Microsoft.EntityFrameworkCore.Tools
```

Método 2: Mediante CLI de .NET en Windows Terminal / PowerShell

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Design
dotnet add package Microsoft.EntityFrameworkCore.Tools

# Asegurar que la herramienta global de EF Core esté disponible en Windows
dotnet tool install --global dotnet-ef
```

Nota de compatibilidad: Asegúrese de que la versión de los paquetes instalados coincida con la versión mayor del SDK de .NET instalado (versión 8.0.x para .NET 8).

Paso 4 — Ejecutar el Scaffolding (Database-First) a clases EF Core

El proceso de **Scaffolding** lee el catálogo del sistema en SQL Server (tablas, columnas, tipos de datos, llaves primarias, foráneas e índices) y genera automáticamente las clases de entidad C# y la clase derivada de DbContext.

1. Cadena de conexión para SQL Server en Windows:

En Windows con SQL Server Express o instancia local con autenticación integrada, la cadena de conexión estándar es:

```
"Server=localhost;Database=TienditaDB;Trusted_Connection=True;TrustServerCertificate=True;"
# En caso de utilizar instancia nombrada SQL Express:
"Server=.\SQLEXPRESS;Database=TienditaDB;Trusted_Connection=True;TrustServerCertificate=True;"
```

2. Ejecución desde la Consola del Administrador de Paquetes (PMC):

```
Scaffold-DbContext "Server=localhost;Database=TienditaDB;Trusted_Connection=True;TrustServerCertificate=True;" Microsoft.EntityFrameworkCore.SqlServer -OutputDir Models -Context TienditaDbContext -Force
```

3. Ejecución alternativa desde Windows PowerShell / CLI de .NET:

```
dotnet ef dbcontext scaffold "Server=localhost;Database=TienditaDB;Trusted_Connection=True;TrustServerCertificate=True;" Microsoft.EntityFrameworkCore.SqlServer --output-dir Models --context TienditaDbContext --force
```

Explicación detallada de los parámetros del comando:

Parámetro	Descripción y utilidad en la práctica
"Server=...;"	Cadena de conexión al motor SQL Server. Incluye TrustServerCertificate=True para evitar errores TLS locales.
Microsoft.EntityFrameworkCore...	Indica el proveedor de base de datos a utilizar para interpretar el esquema.
-OutputDir Models	Crea la carpeta Models dentro del proyecto y almacena allí las clases de entidad y el contexto.
-Context TienditaDbContext	Define el nombre exacto de la clase que hereda de DbContext (por defecto usaría el nombre de la BD).
-Force	Permite sobrescribir los archivos generados en caso de repetir el comando tras cambios en SQL.
-DataAnnotations (Opcional)	Genera atributos ([Table], [MaxLength]) en lugar de mapear todo en OnModelCreating.

¡Importante sobre certificados en Windows! A partir de Microsoft.Data.SqlClient 4.0+, la conexión requiere cifrado por defecto. El parámetro TrustServerCertificate=True; es obligatorio en entornos de desarrollo local para evitar la excepción "The certificate chain was issued by an authority that is not trusted".

Paso 5 — Inspección y análisis del modelo generado

Abra el **Explorador de soluciones** de Visual Studio. Observará que en la carpeta Models/ se crearon los siguientes archivos C#:

- Categoria.cs, Proveedor.cs, Producto.cs, Venta.cs, DetalleVenta.cs (Entidades POCO).
- TienditaDbContext.cs (Contexto de persistencia que hereda de DbContext).

Inspección de Producto.cs (Detección de Relaciones 1:N y Propiedades de Navegación):

```
namespace TienditaApp.Models;

public partial class Producto
{
    public int ProductoId { get; set; }
    public string CodigoBarras { get; set; } = null!;
    public string Nombre { get; set; } = null!;
    public int CategoriaId { get; set; }
    public int ProveedorId { get; set; }
    public decimal PrecioCompra { get; set; }
    public decimal PrecioVenta { get; set; }
    public int Stock { get; set; }
    public DateTime FechaRegistro { get; set; }
    public bool Activo { get; set; }

    // Propiedades de navegación generadas a partir de las Foreign Keys
    public virtual Categoria Categoria { get; set; } = null!;
    public virtual Proveedor Proveedor { get; set; } = null!;
    public virtual ICollection<DetalleVenta> DetalleVenta { get; set; } = new List<DetalleVenta>();
}
```

Inspección de TienditaDbContext.cs (Mapeo Fluent API):

```
public partial class TienditaDbContext : DbContext
{
    public virtual DbSet<Categoria> Categorias { get; set; }
    public virtual DbSet<Proveedor> Proveedores { get; set; }
    public virtual DbSet<Producto> Productos { get; set; }
    public virtual DbSet<Venta> Ventas { get; set; }
    public virtual DbSet<DetalleVenta> DetalleVentas { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Producto>(entity =>
        {
            entity.HasKey(e => e.ProductoId).HasName("PK_Productos");
            entity.Property(e => e.PrecioVenta).HasPrecision(10, 2);
            entity.HasOne(d => d.Categoria).WithMany(p => p.Productos)
                .HasForeignKey(d => d.CategoriaId);
        });
        // Mapeo del resto de relaciones e índices...
    }
}
```

Punto clave de arquitectura: Observe cómo EF Core nombró las propiedades en singular y las colecciones en plural según la convención C#, detectando la relación entre Productos y DetalleVentas con integridad referencial.

Paso 6 — Desacoplar la cadena de conexión a appsettings.json

Al examinar `TienditaDbContext.cs`, encontrará un aviso en el método `OnConfiguring` que advierte no dejar credenciales quemadas en el código fuente. Procedemos a desacoplarla:

1. Crear el archivo `appsettings.json` en la raíz del proyecto `TienditaApp`:

```
{
  "ConnectionStrings": {
    "TienditaConnection": "Server=localhost;Database=TienditaDB;Trusted_Connection=True;TrustServerCertificate=True;"
  }
}
```

En las propiedades del archivo `appsettings.json` en Visual Studio, configure: **Copiar en el directorio de salida: Copiar si es posterior.**

Paso 7 — Prueba de funcionamiento: Consultas LINQ en Program.cs

Reemplazamos el contenido de `Program.cs` para verificar la conexión, consultar el catálogo con `Include` y registrar una transacción real:

Program.cs (Consultas y Pruebas en Vivo):

```
using Microsoft.EntityFrameworkCore;
using TienditaApp.Models;

Console.WriteLine("=== SISTEMA TIENDITA - ENTITY FRAMEWORK CORE 8 ===");
using var db = new TienditaDbContext();

// 1. CONSULTA LINQ CON EAGER LOADING (.Include)
Console.WriteLine("
--- LISTADO DE PRODUCTOS EN STOCK ---");
var productos = db.Productos
    .Include(p => p.Categoria)
    .Include(p => p.Proveedor)
    .Where(p => p.Activo)
    .OrderBy(p => p.Categoria.Nombre)
    .ToList();

foreach (var p in productos)
{
    Console.WriteLine($"{p.Categoria.Nombre} | {p.Nombre,-30} | S/ {p.PrecioVenta,6:F2} | Stock: {p.Stock,3} | Pro
v: {p.Proveedor.RazonSocial}");
}

// 2. RESUMEN DE VENTAS HISTÓRICAS
Console.WriteLine("
--- RESUMEN DE VENTAS ---");
var ventas = db.Ventas
    .Include(v => v.DetalleVenta)
    .ThenInclude(d => d.Producto)
    .ToList();

foreach (var v in ventas)
{
    Console.WriteLine($"Boleta: {v.NumeroBoleta} | Cliente: {v.ClienteNombre} | Total: S/ {v.Total:F2} | Ítems: {v
.DetalleVenta.Count}");
}
```

3. Inserción de una nueva Venta completa usando el Change Tracker:

```
// Agregar una venta con sus detalles en una sola transacción transaccional
var nuevaVenta = new Venta
{
    NumeroBoleta = $"B001-{DateTime.Now.Ticks.ToString() [^8...]}",
    FechaVenta = DateTime.Now,
    ClienteNombre = "Estudiante C# Windows",
    MetodoPago = "Yape",
    Total = 10.50m,
    Estado = "Completada",
    DetalleVenta = new List<DetalleVenta>
    {
        new DetalleVenta { ProductoId = 1, Cantidad = 3, PrecioUnitario = 3.50m, Subtotal = 10.50m }
    }
};

db.Ventas.Add(nuevaVenta);
db.SaveChanges();
Console.WriteLine($"
[OK] Venta registrada con éxito. ID generado por SQL Server: {nuevaVenta.VentaId}");
```

Salida esperada en la consola de Windows:

```
=== SISTEMA TIENDITA - ENTITY FRAMEWORK CORE 8 ===

--- LISTADO DE PRODUCTOS EN STOCK ---
[Abarrotes y Granos] Aceite Primor Clásico 900ml | S/ 8.90 | Stock: 30 | Prov: ALICORP S.A.A.
[Abarrotes y Granos] Arroz Costeño Extra 1kg | S/ 4.80 | Stock: 50 | Prov: ALICORP S.A.A.
[Bebidas y Refrescos] Inca Kola 500ml Pet | S/ 3.50 | Stock: 48 | Prov: ARCA CONTINENTAL...
[Lácteos y Derivados] Leche Gloria Azul 400g Lata | S/ 4.20 | Stock: 72 | Prov: LECHE GLORIA S.A.

[OK] Venta registrada con éxito. ID generado por SQL Server: 5
```

3. Retos adicionales

- **Reto 1 (Actualización de esquema):** Modifique la tabla `dbo.Categorias` agregando una columna `Color` `NVARCHAR(20)` `NULL` en SSMS. Ejecute el comando `Scaffold-DbContext` únicamente para la tabla `Categorias` usando el argumento `-Tables dbo.Categorias -Force` y observe los cambios.
- **Reto 2 (Consultas avanzadas):** Elabore una consulta LINQ que agrupe las ventas por `MetodoPago` y muestre el total recaudado por cada uno (utilizando `GroupBy` y `Sum`).
- **Reto 3 (Inyección de dependencias):** Configure el contexto `TienditaDbContext` usando `Microsoft.Extensions.DependencyInjection` para inyectarlo como servicio *Scoped* en una arquitectura por capas.

4. Entregables y Criterios de Evaluación

Criterio	Evidencia esperada	Puntaje
Base de Datos	5 tablas creadas y pobladas en SQL Server mediante los scripts proporcionados.	25%
Scaffolding	Generación correcta de la carpeta <code>Models</code> con el <code>DbContext</code> y las 5 entidades C#.	35%
Consultas LINQ	Ejecución satisfactoria de consultas con <code>Include</code> y registro de ventas desde C#.	30%
Buenas prácticas	Cadena de conexión desacoplada a <code>appsettings.json</code> y manejo de excepciones.	10%